GAN系列篇-04.CGAN代码篇: 从“无条件生成”到“条件生成”

接下来我们进行CGAN的实现。CGAN在GAN的基础上引入条件变量 c，实现可控的条件生成。

---

1. 实现的策略

根据理论篇的内容可知，CGAN的目标函数为:

$$\min_G \max_D L(D,G) = \mathbb{E}_{x\sim p_{data}(x|c)}\left[\log D(x|c)\right] + \mathbb{E}_{z\sim p_z(z)}\left[\log\left(1-D(G(z|c)|c)\right)\right] \quad (1)$$

在实现时，我们将这个极小-极大博弈分解为两个独立的优化问题:

1.1 判别器的损失函数（D-step）

在固定生成器 G 的情况下，判别器的目标是最大化:

$$L(D) = \mathbb{E}_{x\sim p_{data}(x|c)}[\log D(x|c)] + \mathbb{E}_{z\sim p_z(z)}[\log(1-D(G(z|c)|c))]$$

这等价于最小化二分类交叉熵损失（BCE Loss）:

$$\text{Loss}_D = -\frac{1}{N}\sum_{i=1}^{N}[\log D(x_i|c_i) + \log(1-D(G(z_i|c_i)|c_i))]$$

其中:

- x_i 是真实样本，标签为 1，条件为 c_i
- $G(z_i|c_i)$ 是生成样本，标签为 0，条件为 c_i
- **关键**:判别器不仅要判断样本是否真实，还要判断样本是否与条件匹配

1.2 生成器的损失函数（G-step）

在固定判别器 D 的情况下，生成器的目标是让判别器把假样本误认为真样本（在给定条件下）。

理论上，生成器希望最小化:

$$\text{Loss}_G = \frac{1}{N}\sum_{i=1}^{N}\log(1 - D(G(z_i|c_i)|c_i))$$

但这个目标在训练初期梯度会很小（因为 $D(G(z|c)|c) \approx 0$，所以 $\log(1-D) \approx 0$，梯度接近 0，导致梯度消失）。

因此，实践中通常将生成器的目标改为最大化:

$$\mathbb{E}_{z\sim p_z(z)}[\log D(G(z|c)|c)]$$

这等价于最小化:

$$\text{Loss}_G = -\frac{1}{N}\sum_{i=1}^{N}\log D(G(z_i|c_i)|c_i)$$

且在训练早期能提供更强的梯度。

⚠️ **注意**:这是工程上的改进，两种形式在理论上等价，但在实际训练中，最大化 $\log D(G(z|c)|c)$ 能提供更好的梯度信号。

1.3 训练策略

CGAN的训练采用交替优化的策略:

1. **训练判别器**:固定生成器，更新判别器参数，使其能更好地区分真实样本和生成样本，并判断样本是否与条件匹配
2. **训练生成器**:固定判别器，更新生成器参数，使其能生成与条件匹配的逼真样本

在实际实现中，通常会让判别器训练 k 步（$k \geq 1$），然后训练生成器 1 步，以保持训练的平衡。

1.4 条件编码

条件变量 c 需要编码成数值向量才能输入神经网络。我们使用 **Embedding编码**:

- 方法:使用可学习的嵌入层将类别索引映射为低维向量
- 例 :类别3 → 可学习的N维向量（默认5维）
- 优点:维度可自定义，通常比one-hot更紧凑；可以学习到类别之间的语义关系

---

2. 代码实现

2.1 网络定义

CGAN的网络结构包含两个独立的网络:条件生成器（ConditionalGenerator）和条件判别器（ConditionalDiscriminator）。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ConditionalGenerator(nn.Module):
    """条件生成器:从噪声和标签生成图像"""
    def __init__(self, noise_dim, num_labels, label_embed_dim, hidden_dim,
     output_dim):
        super().__init__()
        self.num_labels = num_labels
        self.label_embed = nn.Embedding(num_labels, label_embed_dim)
        # 输入:噪声 + 标签嵌入
        input_dim = noise_dim + label_embed_dim
        self.linear1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.linear2 = nn.Linear(hidden_dim, hidden_dim * 2)
        self.bn2 = nn.BatchNorm1d(hidden_dim * 2)
        self.linear3 = nn.Linear(hidden_dim * 2, hidden_dim * 4)
        self.bn3 = nn.BatchNorm1d(hidden_dim * 4)
        self.linear4 = nn.Linear(hidden_dim * 4, output_dim)

    def forward(self, z, labels):
        # 标签嵌入
        label_emb = self.label_embed(labels)  # [B, label_embed_dim]
        # 拼接噪声和标签嵌入
        z_cond = torch.cat([z, label_emb], dim=1)  # [B, noise_dim +
        label_embed_dim]
        h = F.leaky_relu(self.bn1(self.linear1(z_cond)), 0.2)
        h = F.leaky_relu(self.bn2(self.linear2(h)), 0.2)
        h = F.leaky_relu(self.bn3(self.linear3(h)), 0.2)
        x = torch.tanh(self.linear4(h))  # 输出范围 [-1, 1]
        return x

class ConditionalDiscriminator(nn.Module):
    """条件判别器:判断输入图像和标签的组合是真实还是生成"""
    def __init__(self, input_dim, num_labels, label_embed_dim, hidden_dim):
        super().__init__()
        self.num_labels = num_labels
        self.label_embed = nn.Embedding(num_labels, label_embed_dim)
        # 输入:图像 + 标签嵌入
        cond_input_dim = input_dim + label_embed_dim
        self.linear1 = nn.Linear(cond_input_dim, hidden_dim * 4)
        self.dropout1 = nn.Dropout(0.3)
        self.linear2 = nn.Linear(hidden_dim * 4, hidden_dim * 2)
        self.dropout2 = nn.Dropout(0.3)
        self.linear3 = nn.Linear(hidden_dim * 2, hidden_dim)
        self.dropout3 = nn.Dropout(0.3)
        self.linear4 = nn.Linear(hidden_dim, 1)
```

```
    def forward(self, x, labels):
        # 标签嵌入
        label_emb = self.label_embed(labels)  # [B, label_embed_dim]
        # 拼接图像和标签嵌入
        x_cond = torch.cat([x, label_emb], dim=1)  # [B, input_dim +
        label_embed_dim]
        h = self.dropout1(F.leaky_relu(self.linear1(x_cond), 0.2))
        h = self.dropout2(F.leaky_relu(self.linear2(h), 0.2))
        h = self.dropout3(F.leaky_relu(self.linear3(h), 0.2))
        out = torch.sigmoid(self.linear4(h))
        return out
```

2.1.1 ConditionalGenerator类的实现

ConditionalGenerator类是一个多层全连接网络，将噪声向量 z 和条件标签 c 映射为图像。前向传播的计算过程 下:

```
def forward(self, z, labels):
    # 1. 标签嵌入:将类别索引转换为嵌入向量
    label_emb = self.label_embed(labels)  # [B, label_embed_dim]

    # 2. 拼接噪声和标签嵌入
    z_cond = torch.cat([z, label_emb], dim=1)  # [B, noise_dim +
        label_embed_dim]

    # 3. 前向传播
    h = F.leaky_relu(self.bn1(self.linear1(z_cond)), 0.2)
    h = F.leaky_relu(self.bn2(self.linear2(h)), 0.2)
    h = F.leaky_relu(self.bn3(self.linear3(h)), 0.2)
    x = torch.tanh(self.linear4(h))  # 输出范围 [-1, 1]
    return x
```

关键设计点:

1. **Embedding编码**:使用 `nn.Embedding` 将类别标签映射为可学习的嵌入向量
2. **条件拼接**:将噪声 z 和标签嵌入 c 拼接后输入网络
3. **Batch Normalization**:在生成器的每一层（除了输出层）使用批归一化，有助于稳定训练
4. **LeakyReLU**:使用 LeakyReLU（负斜率 0.2）代替 ReLU，避免梯度消失问题
5. **Tanh 输出**:最后一层使用 `tanh` 激活函数，将输出限制在 $[-1, 1]$ 范围内

2.1.2 ConditionalDiscriminator类的实现

ConditionalDiscriminator类也是一个多层全连接网络，将图像 x 和条件标签 c 映射为一个标量概率值。前向传播的计算过程 下:

```
def forward(self, x, labels):
    # 1. 标签嵌入:将类别索引转换为嵌入向量
    label_emb = self.label_embed(labels)  # [B, label_embed_dim]

    # 2. 拼接图像和标签嵌入
    x_cond = torch.cat([x, label_emb], dim=1)  # [B, input_dim +
        label_embed_dim]

    # 3. 前向传播
    h = self.dropout1(F.leaky_relu(self.linear1(x_cond), 0.2))
    h = self.dropout2(F.leaky_relu(self.linear2(h), 0.2))
    h = self.dropout3(F.leaky_relu(self.linear3(h), 0.2))
    out = torch.sigmoid(self.linear4(h))  # 输出 [0, 1] 的概率值
    return out
```

关键设计点:

1. **Embedding编码**:同样使用 `nn.Embedding` 进行标签嵌入
2. **条件拼接**:将图像 x 和标签嵌入 c 拼接后输入网络
3. **Dropout**:在判别器的每一层使用 Dropout（0.3），防止过拟合
4. **LeakyReLU**:同样使用 LeakyReLU 激活函数
5. **Sigmoid 输出**:最后一层使用 `sigmoid` 激活函数，输出 $[0, 1]$ 范围内的概率值，表示输入在给定条件下是真实样本的概率

2.2 训练过程

CGAN的训练过程包含两个交替的步骤:

```
import torch.nn as nn
import torch.optim as optim

# 损失函数:二分类交叉熵（用于判别器）
criterion = nn.BCELoss()

# 优化器
g_optimizer = optim.Adam(generator.parameters(), lr=learning_rate,
        betas=(0.5, 0.999))
d_optimizer = optim.Adam(discriminator.parameters(), lr=learning_rate,
        betas=(0.5, 0.999))

# 训练循环
for epoch in range(epochs):
    for images, labels in train_loader:
        images = images.view(-1, input_dim).to(device)  # [B, 784]
        labels = labels.to(device)  # [B]

        # 真实标签和虚假标签
        real_labels = torch.ones(len(images), 1, device=device)
```

```
        fake_labels = torch.zeros(len(images), 1, device=device)

        # ========== 训练判别器 n_critic 次 ==========
        d_loss_batch = 0.0
        for _ in range(n_critic):
            d_optimizer.zero_grad()

            # 真实图像的损失（使用真实条件标签）
            real_output = discriminator(images, labels)
            d_loss_real = criterion(real_output, real_labels)

            # 生成虚假图像（使用真实条件标签）
            noise = torch.randn(len(images), noise_dim, device=device)
            fake_images = generator(noise, labels)

            # 虚假图像的损失（注意:使用 detach() 断开梯度）
            fake_output = discriminator(fake_images.detach(), labels)
            d_loss_fake = criterion(fake_output, fake_labels)

            # 判别器总损失
            d_loss = d_loss_real + d_loss_fake
            d_loss.backward()
            d_optimizer.step()

            d_loss_batch += d_loss.item()

        # ========== 训练生成器 1 次 ==========
        g_optimizer.zero_grad()

        # 生成虚假图像（使用真实条件标签）
        noise = torch.randn(len(images), noise_dim, device=device)
        fake_images = generator(noise, labels)

        # 生成器希望判别器认为虚假图像是真实的（在给定条件下）
        fake_output = discriminator(fake_images, labels)  # 注意:这里不使用
        detach()
        # 使用非饱和 loss:-log(D(x_fake|c))
        fake_output_clamped = torch.clamp(fake_output, min=1e-8, max=1.0)
        g_loss = -torch.mean(torch.log(fake_output_clamped))

        g_loss.backward()
        g_optimizer.step()
```

关键点说明:

1. **判别器训练**:
   - 对真实图像，希望输出接近 1（ `real_labels` ），且条件匹配
   - 对生成图像，希望输出接近 0（ `fake_labels` ），即使条件匹配
   - 使用 `fake_images.detach()` 断开梯度，因为此时只训练判别器
   - **重要**:判别器训练 `n_critic` 次（通常为 1-5），让判别器训练更充分

2. **生成器训练**:
   - 生成器希望判别器认为生成的图像在给定条件下是真实的
   - **使用非饱和损失（non-saturating loss）**：`-log(D(x_fake|c))` 而不是 BCE loss
   - 这样可以避免梯度消失问题，训练更稳定
   - 对 `fake_output` 进行 `clamp(min=1e-8, max=1.0)` 操作，避免 `log(0)` 的数值稳定性问题
   - 不使用 `detach()`，因为需要反向传播更新生成器参数
3. **条件使用**:
   - 在训练时，我们从数据集中采样 (x, c) 对
   - 生成器使用真实条件标签 c 生成图像，确保生成图像与条件匹配
   - 判别器同时接收图像和条件标签，判断 (x, c) 是否匹配
4. **优化器设置**:
   - 使用 Adam 优化器，$\beta_1 = 0.5, \beta_2 = 0.999$（这是 GAN 训练中的常见设置）

---

3. 结果分析

3.1 Loss曲线

根据理论篇的推导，在最优情况下（$p_g(x|c) = p_{data}(x|c)$），CGAN 的目标函数值为:

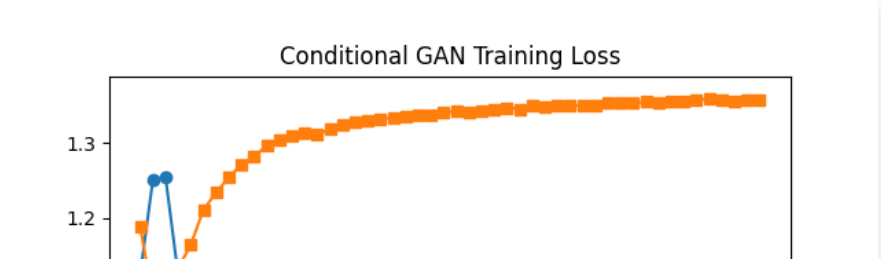
$$L(D^*, G^*) = -2\log 2 = -\log 4 \approx -1.39$$

此时判别器对真实样本和生成样本的输出都接近 0.5，无法区分真假。判别器的 BCE 损失为:

$$\text{Loss}_D = -\log(0.5) - \log(0.5) = 2\log 2 \approx 1.39$$

这与标准GAN的理论值相同，因为CGAN只是在标准GAN的基础上条件化，最优情况下的损失值是一致的。

实际训练50个epochs后的损失曲线显示:



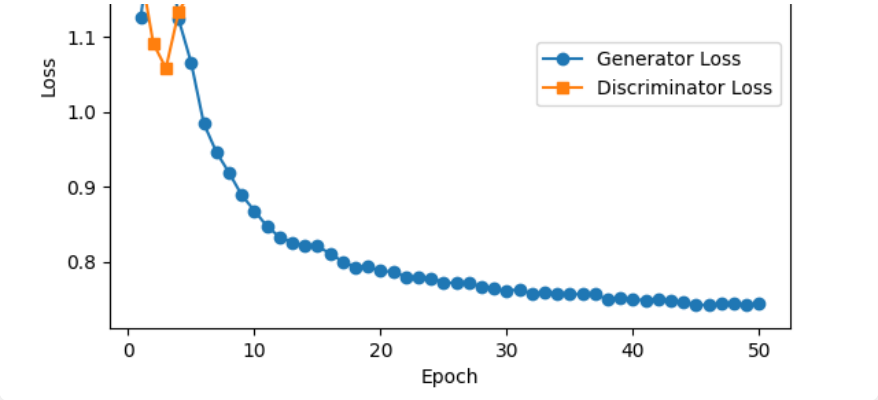


图1 实际训练50个epochs后的损失曲线:展示了条件生成器和条件判别器在实际训练过程中的损失变化。

- **判别器损失**:在训练后期稳定在 1.36 附近，与理论值 $2\log 2 \approx 1.39$ 更加接近
- **生成器损失**:随着训练进行逐渐降低，说明生成器生成的质量不断提升
- **训练稳定性**:损失曲线相对平滑，说明使用非饱和损失、条件信息和 `n_critic` 策略有助于稳定训练

实际训练结果与理论预期高度一致，验证了代码实现的正确性。相比标准GAN，CGAN的训练通常更稳定，因为条件信息为生成器和判别器提供了额外的监督信号。

3.2 生成器:条件生成能力

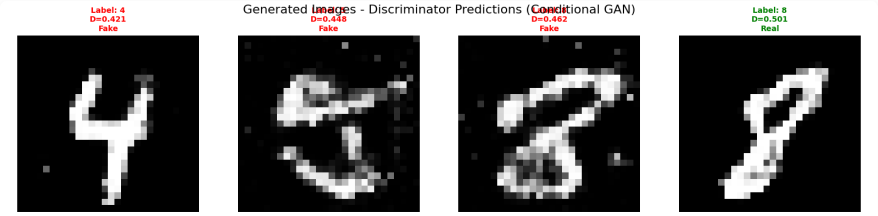
代码中的 `infer` 部分展示了CGAN的**条件生成能力**:

```
# 模型推理 - 条件生成新图像
if infer:
    generator.eval()
    with torch.no_grad():
        # 指定条件标签（例 :生成数字0-9各一张）
        labels = torch.tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], device=device)

        # 从标准正态分布采样噪声向量
        noise = torch.randn(len(labels), noise_dim, device=device)

        # 使用生成器和条件标签生成图像
        fake_images = generator(noise, labels)  # 条件生成
        fake_images = fake_images.view(len(labels), 1, 28, 28)

        # 将输出从 [-1, 1] 转换回 [0, 1] 用于显示
        fake_images = (fake_images + 1) / 2
        fake_images = torch.clamp(fake_images, 0.0, 1.0)
```



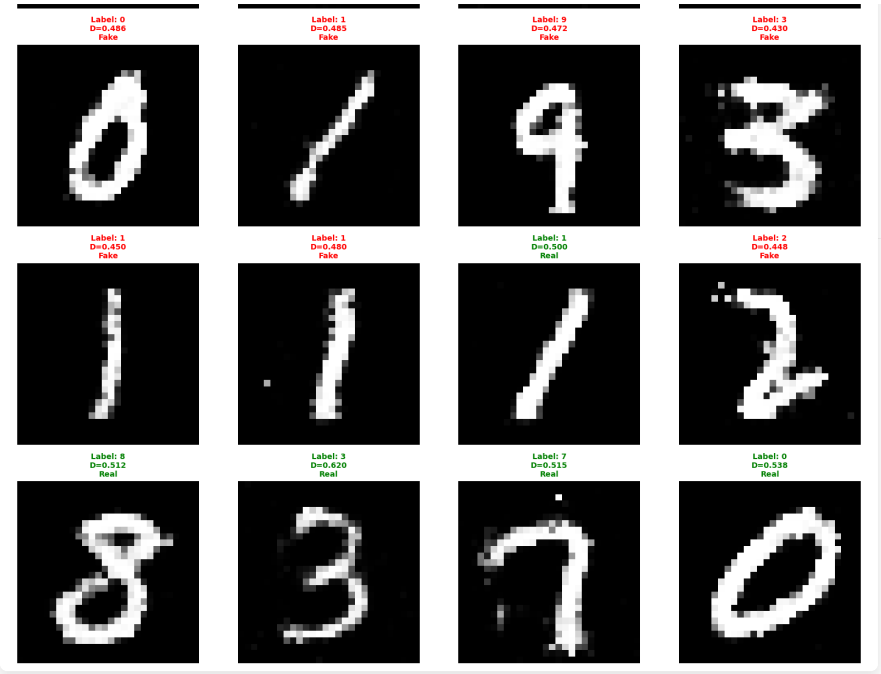


图2 CGAN的条件生成能力:根据指定标签生成的MNIST数字图像

从图2中我们可以得到以下结论:

- **生成质量**:生成器能稳定地生成清晰、多样、类 MNIST 的数字图像
- **条件控制**:生成的图像与指定的条件标签匹配，实现了可控生成

3.3 判别器:判断能力

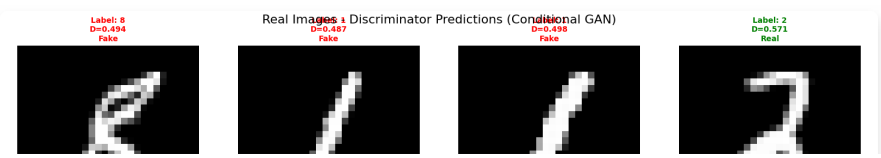
代码中还展示了判别器对真实图像和生成图像的判断能力:

```
# 可视化:真实图像和生成图像的混合，观察判别器的判断
generator.eval()
discriminator.eval()

# 获取真实图像和对应标签
real_image, real_label = next(iter(train_loader))
real_image_flat = real_image[0:1].view(1, -1).to(device)
real_label_tensor = real_label[0:1].to(device)

# 生成一张新图像（使用相同标签）
noise = torch.randn(1, noise_dim, device=device)
fake_image_flat = generator(noise, real_label_tensor)

# 测试判别器的输出
real_pred = discriminator(real_image_flat, real_label_tensor)
fake_pred = discriminator(fake_image_flat, real_label_tensor)
```



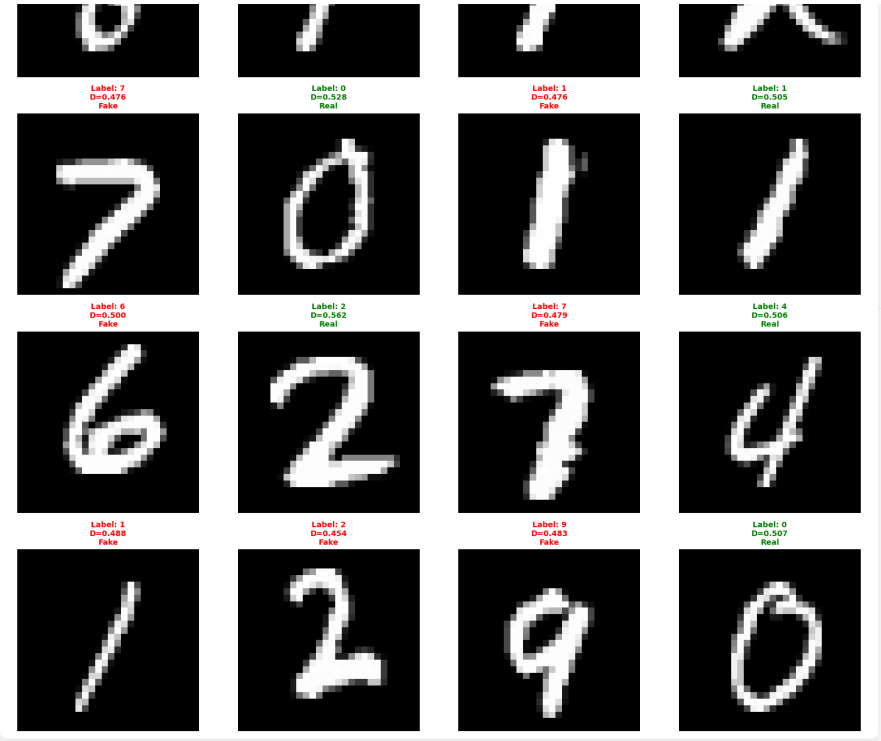


图3 判别器对真实图像和生成图像的判断:展示了16张真实图像和16张生成图像及其对应的判别器输出值

根据图2和图3的判别器输出统计:

- **真实图像（图3）**:平均输出为 **0.5009**，说明判别器对真实图像的平均判断接近 0.5
- **生成图像（图2）**:平均输出为 **0.4855**，说明判别器对生成图像的平均判断也接近 0.5
- **两者差异**:|0.5009 - 0.4855| = 0.0154，差异很小，说明判别器难以明确区分真假图像

---

4. CGAN vs 标准GAN对比总结

特性	标准GAN	CGAN
生成器输入	z	z, c
判别器输入	x	x, c
生成控制	无	可控（由 c 决定）
损失函数	$\log D(x) + \log(1 - D(G(z)))$	$\log D(x\|c) + \log(1 - D(G(z\|c)\|c))$
条件编码	无	Embedding编码

特性	标准GAN	CGAN
应用	无条件生成	条件生成、图像合成、数据增强等

5. 总结

条件生成对抗网络（CGAN） 通过以下关键创新实现了可控生成:

1. **条件对抗**:形成条件对抗，生成器生成与条件匹配的样本，判别器判断样本是否真实且与条件匹配
2. **条件生成**:将条件 c 注入到生成器和判别器中，通过指定不同的条件 c，生成不同类别的样本

CGAN的成功证明了**条件信息**对于可控生成的重要性，为后续的条件生成模型（ Pix2Pix、CycleGAN等）奠定了基础。

📚 参考

- Mirza, M., & Osindero, S. (2014). Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*.
- Goodfellow et al., *Generative Adversarial Nets*, NeurIPS 2014.